XAUUSD Trading & Investment Platform

Detailed Technical & Product Proposal

Core concept: Ultra-fast XAUUSD pending-order trading engine combined with a secure user investment dashboard and full administrator control.

Document scope: Strategy engine, order management, high-speed execution, risk controls, backend, authentication, investment accounting, dashboards, monitoring and phased development.

1. Executive Summary

The proposed product is a complete trading and investment-management platform built around an XAUUSD strategy using **Buy Stop and Sell Stop pending orders**, not market orders. The trading engine will prioritize precise calculations, fast order management, low execution latency, reliable broker confirmations and strict risk controls.

A separate web platform will provide user registration, secure authentication, deposits, investment records, performance tracking, trading history and account information. The administrator will have full operational visibility and control over users, investments, strategy configuration, trading status, risk limits and audit logs.

2. High-Level Architecture

The system should be separated into a high-performance trading layer and a normal web/business layer so that website traffic does not unnecessarily sit inside the critical trading-execution path.

```
WEB PLATFORM → ASP.NET CORE API → PostgreSQL / Redis → Trading Events → C++ TRADING ENGINE → BROKER → XAUUSD
```

3. Trading Strategy Core

The strategy uses a fixed price range with pending orders at the two range boundaries. When one side is triggered, the system manages the opposite pending order and continues the predefined progressive lot sequence.

Example: XAUUSD range 4336.00–4337.00, assuming the broker's point/tick specification supports the stated increments.

Sequence	Price	Order	Lot
1	4337.00	Buy Stop	0.01
2	4336.00	Sell Stop	0.02
3	4337.00	Buy Stop	0.03
4	4336.00	Sell Stop	0.04
5	4337.00	Buy Stop	0.05
6	4336.00	Sell Stop	0.06

Lot progression can be represented as $L(n) = L0 + (n–1) \times \Delta L$. For example, $L0 = 0.01$ and $\Delta L = 0.01$. The actual implementation must validate minimum lot, maximum lot and lot-step requirements from the broker.

4. Ultra-Fast Order Management

The order manager is a critical component. It will calculate, validate, place, confirm, monitor and cancel pending orders with deterministic state tracking.

```
Calculate → Validate → Check symbol specification → Check spread/margin/risk → Send pending order → Confirm broker response → Store ticket → Monitor → Detect trigger → Immediately calculate next order.
```

The system must avoid duplicate orders. Failed requests will use controlled retry logic only after re-checking connection, price, broker state and current order state.

5. Precision & Execution

Price calculations should use broker tick/point rules rather than unsafe assumptions about decimal arithmetic. The engine should load and validate digits, tick size, tick value, contract size, minimum volume, maximum volume and volume step.

For every execution, record requested price, actual fill price, slippage, broker response and execution timing. This creates real execution statistics rather than relying on theoretical assumptions.

6. Basket Profit Engine

```
The primary exit decision should be based on the complete basket rather than treating every position independently.
```

Net Basket P/L = Gross Floating/Realized P/L – **Commission** – **Spread Cost** – **Swap** – **Actual/Estimated Slippage**

When Net Basket P/L reaches the configured target, the engine can close the basket and cancel remaining pending orders, subject to the configured execution and risk rules.

7. Slippage & Cost Model

The platform should record actual slippage after execution. For pre-trade decisions, a conservative configurable slippage buffer can be used. Historical statistics should include average slippage, maximum slippage, spread and execution time.

8. Risk Engine

Because progressive lot sizing can increase exposure rapidly, unlimited order growth should not be allowed. The risk engine should support maximum orders, maximum lot, maximum basket loss, maximum drawdown, maximum margin usage, maximum spread and maximum permitted slippage.

When a hard limit is reached, the engine can stop creating new orders and generate an alert. Risk parameters should be versioned and audited.

9. Desktop Trading Application

A dedicated desktop trading application will provide the operator with live system status without placing the web dashboard inside the critical execution path.

Metric	Example
Symbol	XAUUSD
Strategy	ACTIVE
Range	4336.00–4337.00
Orders	7
Current Lot	0.07
Basket P/L	+$42.18
Net P/L	+$39.62
Spread	Live broker value
Execution latency	Live measurement

10. User Authentication & Accounts

Users should be able to register and securely sign in using email/password and an external provider such as Google. Email verification, password reset, optional phone verification, two-factor authentication and session/device management should be supported.

11. User Dashboard

After login, users receive a simple dashboard showing investment value, current value, profit/loss, return percentage, bot status, performance charts, transaction history and relevant account information.

Suggested sections: Overview, My Investment, Performance, Trade History, Deposits, Withdrawals, Transactions, Bot Status, Profile and Security.

12. Investment & Accounting Flow

```
Deposit → Verification → Investment balance credited → Allocation to trading system → Trading performance → User dashboard.
```

The platform should use a proper transaction ledger rather than simply overwriting a balance. Ledger entries can include deposits, withdrawals, investment allocations, profit, loss, fees, adjustments and refunds.

13. Admin Dashboard — Full Access

The administrator will have complete operational visibility over users, investments, trading accounts, strategy configuration, performance, deposits, withdrawals, system status, alerts and audit logs.

Admin controls should include user management, verification, strategy parameters, risk limits, engine status, account status, transaction review and reporting.

14. Strategy Configuration

Configurable parameters can include symbol, range size, initial lot, lot increment, maximum lot, maximum orders, basket profit target, maximum basket loss, spread limit, slippage limit and trading schedule.

Live changes should follow Draft → Validate → Preview → Confirm → Apply, with a complete audit trail.

15. Database Architecture

Recommended database: PostgreSQL. Core entities include Users, Roles, Sessions, KYC records, Wallets, Transactions, Investments, Investment Allocations, Trading Accounts, Strategies, Strategy Configurations, Orders, Positions, Order Events, Basket Runs, Performance Snapshots, Deposits, Withdrawals, Audit Logs and System Alerts.

16. Real-Time Dashboard

SignalR/WebSockets can provide near-real-time updates for P/L, equity, bot status, orders and performance. The trading engine should continue operating independently if the web dashboard temporarily becomes unavailable.

17. Security

Minimum security controls: HTTPS, secure password hashing, email verification, 2FA, role-based authorization, rate limiting, audit logs, encryption of sensitive data, database backups, secret management, firewalling, secure API authentication and strict separation of broker credentials from the frontend.

18. Monitoring & Alerts

Monitor CPU, RAM, network latency, broker latency, execution latency, order rejections, slippage, spread, engine connectivity and database health. Alerts can be sent through email, Telegram and the admin dashboard.

19. Technology Stack

Component	Recommended Technology
Trading Engine	C++
Broker Integration	Broker/MT5 interface as applicable
Backend API	ASP.NET Core / C#
Database	PostgreSQL
Cache	Redis
Real-time	SignalR / WebSockets
Frontend	React / Next.js
Authentication	ASP.NET Core Identity
Desktop UI	Qt/C++ or WPF/WinUI
Monitoring	Prometheus/Grafana or equivalent
Infrastructure	Dedicated trading server + cloud/web infrastructure

20. Development Roadmap

Phase	Deliverable
1	Mathematical Strategy Engine: range, pending orders, lot progression, basket P/L, costs, slippage and risk rules.
2	Tick-data Backtester: historical testing, drawdown, worst sequences, net P/L and risk statistics.
3	Live Demo Engine: real-time market data and demo pending-order execution.
4	Production C++ Engine: optimized order manager, broker connectivity, monitoring and fail-safes.
5	Web Platform: authentication, user dashboard, deposits, withdrawals, investments, history and admin panel.
6	Full Integration: investment accounting, trading performance, real-time dashboard, alerts and production hardening.

21. Recommended Development Order

Math → Backtester → Demo Execution → C++ Live Engine → Backend → User Dashboard → Investment/Accounting → Production.

The strategy's profitability should be treated as a test result rather than an assumption. Progressive lot sizing can produce positive basket outcomes on some price paths, while one-directional moves, spread, slippage and margin requirements can create large losses. Therefore, extensive tick-level testing and hard risk limits are mandatory before live capital is used.

22. Regulatory & Business Consideration

If users deposit real money and the platform manages or trades that money on their behalf, the product may fall within financial services, investment management, AML/KYC, custody, payments and licensing requirements depending on the jurisdiction and business structure. These requirements should be reviewed with qualified local legal/compliance professionals before production launch.

23. Final Product Vision

The final product is intended to combine an ultra-fast, precision-focused XAUUSD pending-order engine with a professional investment-management interface. The administrator receives full operational control and visibility, while each user receives a secure account where they can deposit funds through the supported process and monitor their investment, performance and history.

Important: Broker specifications, execution rules, contract size, tick size/value, commission, swap, spread and available API/interface must be confirmed before finalizing the mathematical model and live execution implementation.